TRACE: Topology-Aware Visual Analytics for Tree Evolution

Category: Research

Paper Type: please specify

Fig. 1: In the Clouds: Vancouver from Cypress Mountain. Note that the teaser may not be wider than the abstract block.

Abstract—Tree-structured data are common in many domains, such as organizational hierarchies, logical query plans in relational databases, and abstract syntax trees in program analysis. In these applications, tree structures often evolve over time. Analyzing tree evolution is important for understanding how hierarchical structures change, identifying critical transition steps, and diagnosing abnormal or inefficient processes. However, visual analytics of tree evolution remains challenging. First, comparing tree topologies is difficult for analysts, and this challenge becomes even greater in evolution analysis, where they must not only identify structural differences but also understand how one tree transforms into the next. Second, local topological changes are often scattered across large unchanged contexts, making it difficult to highlight critical differences without losing structural context. Third, critical changes in long evolution sequences are usually sparse, making it hard to summarize major trends while preserving key transition steps.

To address these challenges, we propose TRACE, a topology-aware visual analytics framework for the evolution of tree structures. TRACE estimates corresponding changes between adjacent trees, thereby representing the evolution as a sequence of local change units for analysis. It further extracts difference-aware substructures that preserve essential context while reducing visual complexity, supporting effective comparison of critical topological changes. TRACE simplifies long sequences based on the distribution of differences to reveal salient intervals, key transition steps, and overall evolutionary trends. Moreover, TRACE supports multi-granularity analysis from local changes to global evolution patterns.

We evaluate TRACE through case studies on multiple datasets and user interviews. The results show that TRACE helps analysts better understand evolution processes in tree-structured data, locate key structural changes, and identify important patterns and transitions in long sequences.

Index Terms—Radiosity, global illumination, constant time

1 Introduction

Tree-structured data are widely used to represent hierarchical information and model complex processes, such as organizational hierarchies, query plan trees in relational databases, reasoning trees in search and inference tasks, and abstract syntax trees (ASTs) in program analysis. In many real-world scenarios, however, these trees are not static. They evolve over time or across steps as rules are applied, states are updated, search progresses, or human edits are introduced, forming sequences of tree transformations. Analyzing tree evolution is important for understanding how hierarchical structures change, identifying critical transition steps, and diagnosing abnormal or inefficient processes.

Compared with tree sequence analysis that treats trees primarily as a series of snapshots, Tree Evolution analysis is concerned not only with what each tree looks like at a given step, but also with how one tree transforms into the next and why these changes occur. In many applications, the final tree only reflects the result of an evolving process, while obscuring key rewrites, backtracking behaviors, or local irregularities along the way. For example, in query optimization, an initial logical plan may go through a series of rule-based rewrites and structural adjustments before becoming an executable plan. Examining this process can help analysts understand optimizer behavior and locate the steps that lead to incorrect results or performance degradation. Similarly, in reasoning and search tasks, tree expansion, pruning, and rollback reveal the exploration strategy of the system; in program evolution analysis, local modifications on ASTs and their propagated effects can help expose refactoring behavior and error sources. These examples suggest that trees should often be understood as evolving structures rather than isolated snapshots.

Despite its importance, visual analytics of Tree Evolution remains challenging in at least three aspects. First, prior studies have shown that even comparing relatively simple graphs can be difficult for analysts, because topological comparison often disrupts cognitive continuity. This challenge becomes even greater in Tree Evolution analysis, where analysts must not only identify structural differences but also understand how one tree transforms into the next. Second, local topological changes are often scattered across large unchanged contexts, making it difficult to highlight critical differences without losing the structural

context needed for interpretation. Third, critical changes in long evolution sequences are often sparse, making it hard to summarize major trends while preserving key transition steps. Together, these challenges call for methods that support topology-aware comparison, context-preserving difference representation, and effective summarization of long evolution processes.

Existing studies have explored related problems from several perspectives, including domain-specific evolution analysis, dynamic tree visualization, structural differencing, and temporal visual analytics. Some approaches are designed for specific application domains. For example, QOVIS supports the analysis of query optimizer behaviors by leveraging domain knowledge about query rewriting and optimization processes. While such systems are effective in their target settings, they rely on strong domain assumptions and are difficult to generalize to broader Tree Evolution analysis tasks. Other approaches use animation or dynamic tree views to present transitions between adjacent steps, which can help reveal local continuity but become less effective for reviewing long sequences, locating critical steps, or comparing sparse changes across time. Temporal visual analytics methods can summarize trends and stage-wise patterns, but they often weaken the hierarchical topology and fine-grained change semantics of the original trees. Structural differencing methods can quantify edit relationships between trees, yet they are typically not designed for long-sequence visual analysis and often do not balance structural context preservation with interpretability. Overall, existing approaches still fall short of jointly supporting topology-aware comparison between adjacent trees, context-preserving representation of critical differences, and long-sequence change summarization.

To address this gap, we propose TRACE, a topology-aware visual analytics framework for Tree Evolution. TRACE supports analysis at three levels. At the modeling level, it estimates corresponding changes between adjacent trees, representing evolution as a sequence of analyzable local change units rather than a set of disconnected snapshots. At the structural comparison level, it extracts compact difference-aware substructures that preserve essential context while reducing visual complexity, helping analysts focus on critical topological changes. At the sequence level, it organizes and simplifies local differences over time to reveal salient intervals, key transition steps, and overall evolutionary trends. By combining change modeling, topology-aware difference visualization, and sequence summarization, TRACE supports multi-granularity analysis from local changes to global evolution patterns.

The contributions of this work are as follows:

- We formulate Tree Evolution as a visual analytics problem that jointly involves topology-aware comparison, context-preserving difference analysis, and long-sequence summarization, and we characterize common analytical needs shared across multiple application scenarios.
- We propose a configurable differencing method for adjacent tree structures to estimate corresponding changes and evolution processes, and further develop a compact difference-aware substructure extraction strategy that balances fidelity and cognitive load.
- We design and implement TRACE, a multi-granularity visual analytics framework that supports local topology comparison, cross-step evolution tracing, and long-sequence summarization of key processes.
- We demonstrate the effectiveness and practical value of TRACE through multiple case studies and user interviews in tasks involving tree evolution understanding, key change localization, and long-sequence analysis.

The remaining parts of the paper are organized as follows:

2 Related Work

2.1 Dynamic Tree Visualization

A substantial body of work has studied how to visualize hierarchical structures that evolve over time, with particular emphasis on preserving structural readability and supporting users' mental continuity across timesteps. One representative direction focuses on stable space-filling layouts for evolving hierarchies. Temporal Treemaps transforms a sequence of trees with known correspondences into a static nested treemap, allowing users to inspect appearance, disappearance, merge, and split events without relying solely on animation [12]. EvoCells similarly proposes a treemap layout strategy tailored to evolving tree data, explicitly mapping tree updates to rectangle updates in order to improve layout stability under hierarchy changes [17]. Stable Voronoi treemap approaches further seek to reduce unnecessary layout variation when hierarchies evolve, thereby improving cross-time comparability while preserving the overall spatial organization of the hierarchy [10].

Another line of work explores static or semi-static representations that directly encode hierarchy changes across multiple steps. Dynamic indented plots visualize changes between successive hierarchies using time-to-space mappings and explicit visual cues for structural updates, making it possible to compare multiple timesteps in a single view [4]. TreeVersity2 constructs dynamic hierarchies and supports change exploration through space-filling visualizations and multilevel summaries, enabling users to inspect created, removed, and modified nodes over time [9]. Beyond tree-specific visualizations, Vehlow et al. proposed a matrix-based visualization that combines hierarchical grouping information, temporal ordering, and flow-based transition cues to support the analysis of dynamic hierarchies in evolving graph sequences [21]. These methods are effective for presenting temporal continuity and preserving structural readability, but they are primarily designed for showing how hierarchies change across time rather than for explicitly modeling how one tree transforms into the next. In addition, when evolution sequences are long and critical changes are sparse, animation- or snapshot-based views often make it difficult to review distant steps, locate salient transitions, or focus on topology-aware differences while retaining essential structural context.

2.2 Tree Differencing and Change Modeling

Another relevant line of research focuses on computing structural differences between trees. The classical tree edit distance formulation introduced by Zhang and Shasha established a foundational model for computing the minimum-cost sequence of insert, delete, and relabel operations required to transform one tree into another [29]. This formulation has had lasting influence because it provides a principled way to describe structural differences and change processes between two trees. Later work improved the efficiency and robustness of tree edit distance computation. RTED introduced a robust strategy selection mechanism that performs well across different tree shapes [15], while AP-TED further improved efficiency and scalability for practical tree comparison [16]. Approximate or alternative similarity measures, such as pq-Gram distance, have also been proposed to make tree comparison more scalable for large datasets [1].

Beyond generic tree edit distance, several methods address differencing in specific classes of structured data. X-Diff detects changes in XML documents by treating them as unordered trees and generating change descriptions based on structural matching [24]. Earlier work on change detection in hierarchically structured information argued that meaningful change descriptions may require richer structural operations and explicit matching strategies [5]. Tree differencing has also been extensively studied in source code analysis, where abstract syntax trees provide a natural hierarchical representation of programs. ChangeDistiller extracts fine-grained structural edits from ASTs and organizes them into semantically meaningful change types [8]. GumTree further improves AST differencing by combining top-down and bottom-up matching strategies to recover node correspondences and edit scripts that better reflect developer intent, including move operations [7]. More specialized topology-driven differencing methods, such as branch-based edit distances for merge trees, show how structural comparison can be adapted to topology-centric settings in scientific visualization [3].

Although this literature provides an essential computational foundation, it does not by itself solve the visual analytics problem addressed in our work. Most tree differencing methods are designed to compute distances, correspondences, or edit scripts, rather than to support human-centered analysis of tree evolution. They are therefore less concerned with how differences should be presented to analysts, how

unchanged context should be preserved to maintain interpretability, or how local structural edits should be integrated into the analysis of long evolution sequences. These methods are powerful for modeling structural change, but they are not sufficient for topology-aware visual comparison and multi-step evolution analysis.

2.3 Temporal Summarization and Sequence Abstraction

Our work is also related to visual analytics research on temporal sequence summarization. A large body of work in this area studies how to abstract long or complex event sequences into more compact representations that reveal high-level patterns, salient intervals, and representative stages. LifeFlow introduced an overview representation for event sequences that aggregates many temporal records into aligned visual timelines, making it easier to identify common pathways, temporal spacing, and outcome-related patterns [26]. EventFlow extended this direction by providing simplification operations and a visual complexity model, enabling analysts to progressively reduce clutter and discover important temporal structures in large collections of sequences [14]. Outflow uses flow-based visual representations to summarize temporal pathways and transitions across many event sequences [23], while DecisionFlow addresses high-dimensional temporal event data through milestone-based abstractions and interactive multi-view exploration [27].

Subsequent work further explored multilevel abstraction, stage analysis, and progression summarization for complex temporal data. EventThread and ET2 focus on stage analysis, temporal segmentation, and progression understanding, providing methods to identify latent stages and summarize how sequences evolve across them [30, 31]. Sequen-C supports multilevel overviews of temporal event sequences, allowing analysts to move between coarse and fine-grained representations while preserving sequence-level context [25]. Other systems, such as SSRDVis and LoLo, emphasize pattern summarization, anomaly detection, and multilevel exploration for long event sequences with many events [11,28]. This literature is highly relevant to Tree Evolution because long evolution processes often contain only a limited number of critical structural changes, while the rest of the sequence mainly provides contextual continuity.

However, most temporal summarization methods operate on event records, categorical timelines, or abstract sequence tokens rather than on hierarchical structures with rich topology. As a result, they typically abstract away the hierarchical organization and fine-grained structural semantics that are central to Tree Evolution analysis. Even when they reveal progression stages effectively, they do not naturally support topology-aware comparison between adjacent trees, nor do they preserve the structural context needed to interpret local changes in a hierarchy. Therefore, while temporal sequence summarization offers important ideas for long-sequence abstraction, additional mechanisms are needed to integrate such abstraction with structural change modeling and visual comparison in evolving trees.

2.4 Domain-Specific Analysis of Evolving Trees

Several studies have demonstrated the practical importance of analyzing evolving tree-like structures in specific domains. In database systems, Picasso visualizes optimizer behavior across query parameter spaces and helps users inspect how query plans vary under different conditions [19]. More recently, QOVIS directly supports understanding and diagnosing query optimization by exposing intermediate query plans, transformation logic, and the optimization process through a visualization-assisted framework [20]. These systems are particularly relevant to our work because query plans are a representative example of evolving tree structures, and because the analysis tasks they support—such as understanding transformation behavior, locating problematic steps, and diagnosing abnormal outcomes—closely match the goals of Tree Evolution analysis.

In software engineering and program analysis, hierarchical program representations also evolve over time. Tree-based version control in Envision explores how AST-aware differencing and merging can support semantically meaningful version management [18]. Related work on dynamic call graphs and call graph evolution further illustrates how tree-like or hierarchical dependency structures can change across program versions or executions, and how visual or analytical methods can help identify recurring patterns, unstable components, or noteworthy structural changes [2, 22]. Tree evolution is also important in search, constraint solving, and scientific data analysis. Visual Search Tree Profiling provides tools for inspecting large search trees, comparing search behaviors, and identifying critical or repeated substructures during search processes [6]. In scientific visualization, Temporal Merge Tree Maps summarizes the evolution of topological features over time by aligning and linearizing merge trees extracted from temporal scalar data [13].

These domain-specific systems provide strong evidence that evolving hierarchical structures are analytically important across many applications. At the same time, they also highlight an important limitation of prior work: their reliance on application-specific assumptions. Query optimizer analysis depends on optimizer rules and plan semantics; AST and call graph analysis depends on programming-language structure and software engineering tasks; search tree profiling depends on domain-specific notions of search behavior and performance; topology-based scientific visualizations depend on properties of scalar fields and feature tracking. These assumptions make such systems highly effective in their target domains, but also limit their generalizability. In contrast, our goal is to develop a general topology-aware visual analytics framework for Tree Evolution that supports corresponding change understanding, context-preserving structural comparison, and long-sequence summarization across different types of tree-structured processes.

Summary. In summary, prior work has contributed important foundations from the perspectives of dynamic hierarchy visualization, tree differencing, temporal sequence abstraction, and domain-specific evolution analysis. Dynamic visualization methods help present changing hierarchies over time, but usually emphasize continuity and readability over explicit transformation understanding. Tree differencing methods provide computational tools for recovering correspondences and edit relationships, but are not designed to support human-centered, context-preserving comparison in long evolution sequences. Temporal summarization methods enable abstraction of long processes, but typically suppress the hierarchical topology and fine-grained structural changes that matter in Tree Evolution analysis. Domain-specific systems demonstrate the real-world importance of evolving trees, but rely on task-specific assumptions that hinder generalization. Our work addresses these limitations by integrating corresponding change modeling, difference-aware substructure extraction, and sequence-level simplification into a unified topology-aware visual analytics framework for Tree Evolution.

3 Data Model and Problem Definition

3.1 Tree, Tree Sequence, and Evolution

We consider a rooted tree as the basic structural unit in our data. A tree is defined as

$$T = (V, E, r),$$

where V is the set of nodes, $E \subseteq V \times V$ is the set of directed parent-child edges, and $r \in V$ is the root node. Each node $v \in V$ may be associated with a label, an attribute vector, or other application-specific metadata. In the following, we mainly focus on the hierarchical structure of the tree, while allowing node attributes to be incorporated when needed.

For a node $v \in V$, we denote by $parent(v)$ its parent node, and by $children(v)$ the set of its child nodes. The subtree rooted at v is denoted by

$$T(v) = (V(v), E(v), v),$$

where $V(v) \subseteq V$ contains v and all its descendants, and $E(v) \subseteq E$ contains the edges induced by $V(v)$.

To model structural evolution over time, we consider not a single tree, but an ordered sequence of trees:

$$\mathscr{T} = \langle T_1, T_2, \ldots, T_n \rangle,$$

where each T_i represents the tree observed at the i-th time step, version, or evolutionary stage. We refer to $\mathscr{T}$ as a *tree sequence*. The ordering

of trees may be induced by time, iteration, version number, or any domain-specific progression. In this work, we assume that the sequence order is meaningful and that structural evolution can be analyzed by comparing adjacent trees in the sequence.

Given a tree sequence $\mathscr{T}$, we define *tree evolution* as the structural transformation process across consecutive trees. Rather than modeling evolution only at the level of the full sequence, we decompose it into a series of local transitions between adjacent trees. For two adjacent trees T_i and T_{i+1}, the transition

$$\delta_i : T_i \to T_{i+1}$$

is referred to as an *evolution step*. Each evolution step captures the structural changes occurring between two consecutive states of the tree sequence.

This adjacency-based formulation has two advantages. First, it enables analysts to inspect evolution incrementally, which is often more cognitively manageable than comparing distant trees directly. Second, it provides a natural basis for identifying local structural changes and tracking how such changes accumulate over the entire sequence. In this work, our primary analytical focus is on the comparison of adjacent trees, i.e., the characterization of each evolution step δ_i.

3.2 Structural Difference and Difference Nodes

To quantify the amount of change in an evolution step, we model the difference between T_i and T_{i+1} using tree edit operations. Specifically, we consider a transformation sequence that converts T_i into T_{i+1} through a set of elementary edit operations, such as node insertion, node deletion, and node relabeling or substitution, depending on the application setting.

The *tree edit distance* between two adjacent trees is defined as the minimum cost required to transform one tree into the other:

$$\mathrm{TED}(T_i, T_{i+1}).$$

This value provides a quantitative measure of structural difference. A larger edit distance indicates a greater amount of structural change between the two adjacent stages.

However, the edit distance alone only reflects *how much* the structure has changed, but does not explicitly reveal *where* the changes occur or how they are structurally related. Therefore, beyond this scalar quantity, we derive a set of structural difference elements from the edit sequence.

Let

$$D_i = \{d_1, d_2, \ldots, d_m\}$$

denote the set of nodes involved in the minimum edit sequence between T_i and T_{i+1}. We refer to these nodes as *difference nodes*. Intuitively, a difference node marks a local position in the tree where a structural modification occurs in the corresponding evolution step. Difference nodes constitute the finest-grained representation of structural change. They preserve the exact locations of local edits and therefore provide maximal detail about the transformation between adjacent trees.

Nevertheless, directly presenting all difference nodes is often insufficient for effective analysis. When the number of changes is large, or when changed nodes are scattered across the hierarchy, a pure node-level representation may impose excessive cognitive burden and obscure higher-level structural patterns. This motivates the need for more structured abstractions built on top of difference nodes.

3.3 Difference Subtrees and Difference Forests

To organize difference nodes into interpretable structural units, we introduce the concepts of *difference subtree* and *difference forest*.

For a tree T and a node $r \in V$, the subtree $T(r)$ is called a difference subtree if it contains at least one difference node. More generally, for a subset of difference nodes $X \subseteq D_i$, if all nodes in X are contained in $T(r)$, then $T(r)$ is said to be a difference subtree for X. Among all such subtrees that cover X, we define the one with the smallest structural extent as the *minimum difference subtree* for X, denoted as

$$T^*(X).$$

Depending on the precise implementation, the structural extent may be measured by the number of nodes, the number of edges, or the induced hierarchical span. In the simplest setting, $T^*(X)$ can be understood as the smallest rooted subtree that covers all nodes in X.

The difference subtree provides a localized structural context for a set of changes. Rather than showing isolated difference nodes one by one, it groups nearby or structurally related differences into a higher-level region, thereby supporting more interpretable qualitative analysis.

In many evolution steps, difference nodes are distributed across multiple disconnected or weakly related regions of the tree. In such cases, a single difference subtree may be too coarse to summarize the changes effectively. We therefore define a *difference forest* as a set of pairwise disjoint difference subtrees that jointly cover a target set of difference nodes. Formally, let $X \subseteq D_i$ be a set of difference nodes. A set of subtrees

$$F = \{T(r_1), T(r_2), \ldots, T(r_k)\}$$

is called a difference forest for X if the following two conditions hold:

$$X \subseteq \bigcup_{j=1}^{k} V(r_j),$$

and

$$V(r_p) \cap V(r_q) = \emptyset, \quad \forall p \neq q.$$

That is, every node in X is covered by at least one subtree in the forest, and any two subtrees in the forest are non-overlapping.

A *minimum difference forest* for X is a difference forest in which each subtree is a minimum difference subtree for the subset of difference nodes it covers. Importantly, for a given set of difference nodes, multiple valid minimum difference forests may exist. This non-uniqueness reflects the fact that structural changes can be summarized at multiple granularities, depending on how one chooses to partition the difference nodes into disjoint structural regions.

Difference forests serve as an intermediate representation between two extremes:

- presenting every difference node individually, which preserves maximal detail but may lead to clutter; and
- summarizing all changes with a single large subtree, which reduces complexity but may obscure local distinctions.

This representation is particularly suitable for visual analysis, as it enables structured summaries of differences at multiple levels of abstraction while preserving a clear, non-overlapping decomposition of changed regions.

3.4 Difference Summarization and Problem Statement

The definitions above establish a hierarchy of difference representations:

- *tree edit distance* captures the overall magnitude of change;
- *difference nodes* capture exact local changes;
- *difference subtrees* capture local structural contexts of change; and
- *difference forests* capture structured summaries of distributed changes.

Based on this hierarchy, we formulate adjacent-tree comparison as a *difference summarization problem*. Given two adjacent trees T_i and T_{i+1}, we aim to derive a representation that is both informative and cognitively manageable.

A trivial solution is to directly show all difference nodes. This preserves all local details but may overwhelm analysts when changes are numerous. Another trivial solution is to aggregate all difference nodes into a single large subtree. This reduces the number of visual elements but may hide important structural details. Therefore, the central challenge is to identify an appropriate difference forest that balances *detail preservation* and *representation simplicity*. This trade-off forms the basis of our method.

Given a tree sequence

$$\mathscr{T} = \langle T_1, T_2, \ldots, T_n \rangle,$$

our task is to analyze each evolution step $\delta_i : T_i \to T_{i+1}$ by jointly supporting:

- *quantitative comparison*, through the tree edit distance $\mathrm{TED}(T_i, T_{i+1})$; and
- *qualitative comparison*, through a structured summary of the corresponding difference nodes.

More specifically, for each adjacent tree pair (T_i, T_{i+1}), we seek to construct a difference forest F_i over the difference node set D_i such that:

- all important structural differences are covered;
- the resulting summary remains structurally interpretable; and
- the complexity of inspecting the changes is reduced without sacrificing excessive detail.

In the next section, we describe how this data model supports the design of our method for constructing and selecting appropriate difference forests.

3.5 Difference Forest Construction Algorithm

Based on the above data model, we formulate difference forest construction as a constrained grouping problem over the set of difference nodes. Given the difference node set D_i, our goal is to construct a difference forest

$$F_i = \{T(r_1), T(r_2), \ldots, T(r_k)\}$$

such that all difference nodes are covered, the subtrees are pairwise disjoint, and the resulting forest achieves a balance between preserving local details and reducing cognitive complexity.

Basic idea. The key idea is to treat each difference node as an initial atomic group and then iteratively merge structurally related groups in a bottom-up manner. Each group of difference nodes is represented by its minimum difference subtree. The algorithm starts from the finest-grained representation, where each difference node forms an individual subtree, and progressively combines groups if the merge leads to a better trade-off between structural fidelity and representational simplicity. The merging process terminates when no valid merge can further improve the objective.

Group representation. Let $G = \{g_1, g_2, \ldots, g_k\}$ denote a partition of the difference node set D_i, where each group $g_j \subseteq D_i$ contains one or more difference nodes, and

$$\bigcup_j g_j = D_i, \qquad g_p \cap g_q = \emptyset \text{ for } p \neq q.$$

For each group g_j, we compute its minimum difference subtree $T^*(g_j)$. The corresponding difference forest is therefore

$$F_i(G) = \{T^*(g_1), T^*(g_2), \ldots, T^*(g_k)\}.$$

A valid forest additionally requires that these subtrees are pairwise disjoint.

Objective function. To evaluate the quality of a difference forest, we define an objective function that balances three criteria: contextual redundancy, intra-subtree dispersion, and forest complexity. Specifically, for a forest F, we define

$$\mathscr{L}(F) = \alpha \cdot C_{\text{extra}}(F) + \beta \cdot C_{\text{disp}}(F) + \lambda \cdot C_{\text{count}}(F),$$

where α, β, and λ are user-controllable weights.

The first term measures the amount of redundant context introduced by the forest:

$$C_{\text{extra}}(F) = \sum_{S \in F} \left(|V(S)| - |D_i \cap V(S)| \right).$$

This term penalizes subtrees that include many non-difference nodes and therefore become overly coarse.

The second term measures how dispersed the difference nodes are within each subtree:

$$C_{\text{disp}}(F) = \sum_{S \in F} \frac{1}{|D_S|} \sum_{d \in D_S} dist(root(S), d),$$

where $D_S = D_i \cap V(S)$ and $dist(root(S), d)$ denotes the tree distance from the root of subtree S to difference node d. This term favors subtrees whose covered difference nodes are structurally compact.

The third term controls the complexity of the forest:

$$C_{\text{count}}(F) = |F|.$$

This term penalizes forests with too many subtrees and helps reduce visual and cognitive fragmentation.

Our goal is to find a forest that minimizes $\mathscr{L}(F)$ subject to the coverage and non-overlap constraints:

$$D_i \subseteq \bigcup_{S \in F} V(S),$$

$$V(S_p) \cap V(S_q) = \emptyset, \quad \forall S_p, S_q \in F,\ p \neq q.$$

Greedy bottom-up optimization. We adopt a greedy bottom-up optimization strategy. Initially, each difference node $d \in D_i$ forms a singleton group $g = \{d\}$, and the initial forest consists of the corresponding minimum difference subtrees. At each iteration, the algorithm considers merging two existing groups g_a and g_b into a new group

$$g_{ab} = g_a \cup g_b.$$

The merged group is represented by the minimum difference subtree $T^*(g_{ab})$.

A merge is considered valid only if the new subtree remains disjoint from all other subtrees currently in the forest. For each valid candidate merge, the algorithm computes the updated forest and evaluates the change in objective value:

$$\Delta\mathscr{L} = \mathscr{L}(F') - \mathscr{L}(F).$$

The algorithm then selects the merge with the largest reduction in objective value. If the best candidate satisfies $\Delta\mathscr{L} < 0$, the merge is accepted and the forest is updated. Otherwise, the process terminates.

This strategy naturally proceeds from fine-grained local differences to coarser structural summaries, and the optimization stops at the point where further abstraction would introduce more loss than benefit.

Algorithm. The procedure is summarized below.

```
Algorithm 1: Difference Forest Construction
Input: rooted tree T, difference node set D_i, weights alpha, b
Output: pairwise disjoint difference forest F

Initialize groups G = { {d} | d in D_i }
Compute F = { T*(g) | g in G }
repeat
    bestDelta <- 0
    bestMerge <- null
    for each pair (g_a, g_b) in G do
        g_ab <- g_a union g_b
        S_ab <- T*(g_ab)
        if S_ab overlaps any subtree in F \ {T*(g_a), T*(g_b
            continue
        end if
        F' <- (F \ {T*(g_a), T*(g_b)}) union {S_ab}
        delta <- L(F') - L(F)
        if delta < bestDelta then
            bestDelta <- delta
            bestMerge <- (g_a, g_b, S_ab)
        end if
    end for
    if bestMerge != null then
        update G and F using bestMerge
    end if
until bestMerge == null
return F
```

Discussion. The proposed algorithm has several desirable properties. First, it is consistent with the hierarchical structure of trees: each merge corresponds to a structurally meaningful aggregation of difference nodes. Second, because disjointness is explicitly enforced during optimization, the resulting forest always satisfies the non-overlap constraint in our definition. Third, the algorithm can naturally produce multi-granularity results: the intermediate forests generated during optimization form a hierarchy of summaries, allowing analysts to move from fine-grained local changes to coarser structural overviews.

In the next section, we build upon this algorithm to design a visual analytics workflow for adjacent-tree comparison and long-sequence evolution analysis.

3.6 Our solutions

References

[1] N. Augsten, M. Böhlen, and J. Gamper. The pq-gram distance between ordered labeled trees. *Journal of Discrete Algorithms*, 2010. 2

[2] M. Bao et al. Call graph evolution analytics over a version series of an evolving software system. In *ASE*, 2022. 3

[3] C. Bauer et al. Branch decomposition-independent edit distances for merge trees. *Computer Graphics Forum*, 2022. 2

[4] M. Burch et al. Visualizing hierarchy changes by dynamic indented plots. In *IEEE VIS*, 2014. 2

[5] S. S. Chawathe, A. Rajaraman, H. Garcia-Molina, and J. Widom. Change detection in hierarchically structured information. In *SIGMOD*, 1996. 2

[6] J. Dubois et al. Visual search tree profiling. *Constraints*, 2015. 3

[7] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus. Fine-grained and accurate source code differencing. In *ASE*, 2014. 2

[8] B. Fluri, M. Wursch, M. Pinzger, and H. C. Gall. Change distilling: Tree differencing for fine-grained source code change extraction. *IEEE Transactions on Software Engineering*, 2007. 2

[9] J. Guerra-Gómez et al. TreeVersity2 – visualizing change over time using dynamic hierarchies. In *EuroVis*, 2013. 2

[10] D. Hahn et al. Visualization of varying hierarchies by stable layout of voronoi treemaps. In *EuroVis*, 2014. 2

[11] J. H. Isenberg et al. LoLo: A visual analytics approach for long sequences with a lot of events. *Information Visualization*, 2025. 3

[12] M. Köpp and T. Weinkauf. Temporal treemaps: Static visualization of evolving trees. *Computer Graphics Forum*, 2018. EuroVis 2018. 2

[13] M. Köpp and T. Weinkauf. Temporal merge tree maps: A topology-based static visualization for temporal scalar data. *IEEE Transactions on Visualization and Computer Graphics*, 2022. 3

[14] M. Monroe, R. Lan, J. Delfi, G. Ellis, C. Plaisant, and B. Shneiderman. Temporal event sequence simplification. In *CHI*, 2013. EventFlow. 3

[15] M. Pawlik and N. Augsten. RTED: A robust algorithm for the tree edit distance. *Information Systems*, 2011. 2

[16] M. Pawlik and N. Augsten. Efficient computation of the tree edit distance. *ACM Transactions on Database Systems*, 2015. AP-TED. 2

[17] L. Scheibel et al. EvoCells: A treemap layout algorithm for evolving tree data. In *International Joint Conference on Computer Vision, Imaging and Computer Graphics*, 2018. 2

[18] W. S. Schneider. Tree-based version control in envision. Master's thesis, Unknown Institution, 2015. 3

[19] S. Sudarshan et al. Picasso: A database query optimizer visualizer. *Proceedings of the VLDB Endowment*, 2010. 3

[20] H. Tian et al. QOVIS: Understanding and diagnosing query optimizer via a visualization-assisted approach. *Proceedings of the VLDB Endowment*, 2025. 3

[21] C. Vehlow, F. Beck, and D. Weiskopf. Visualizing dynamic hierarchies in graph sequences. *IEEE Transactions on Visualization and Computer Graphics*, 2016. 2

[22] E. von Praun et al. Visualizing dynamic call graphs. In *VMV*, 2012. 3

[23] F. Wang et al. Outflow: Visualizing patient flow by flow map layouts. *IEEE Transactions on Visualization and Computer Graphics*, 2014. 3

[24] Y. Wang, D. J. DeWitt, and J.-Y. Cai. X-Diff: An effective change detection algorithm for XML documents. In *ICDE*, 2003. 2

[25] Y. Wang et al. Sequen-C: A multilevel overview of temporal event sequences. *IEEE Transactions on Visualization and Computer Graphics*, 2022. 3

[26] K. Wongsuphasawat, J. A. Guerra Gómez, C. Plaisant, T. Wang, M. Taieb-Maimon, and B. Shneiderman. LifeFlow: Visualizing an overview of event sequences. In *CHI*, 2011. 3

[27] P. Xu et al. DecisionFlow: Visual analytics for high-dimensional temporal event sequence data. *IEEE Transactions on Visualization and Computer Graphics*, 2014. 3

[28] L. Yang et al. SSRDVis: Interactive visualization for sequence summarization and rare behavior detection. *Journal of Visualization*, 2019. 3

[29] K. Zhang and D. Shasha. Simple fast algorithms for the editing distance between trees and related problems. *Algorithmica*, 1989. 2

[30] J. Zhao et al. EventThread: Visual summarization and stage analysis of event sequence data. *IEEE Transactions on Visualization and Computer Graphics*, 2018. 3

[31] J. Zhao et al. Visual progression analysis of event sequence data. *Computer Graphics Forum*, 2018. ET2. 3

A About Appendices

Refer to **??** for instructions regarding appendices.

B Troubleshooting

B.1 ifpdf error

If you receive compilation errors along the lines of `Package ifpdf Error: Name clash, \ifpdf is already defined` then please add a new line `\let\ifpdf\relax` right after the `\documentclass[journal]{vgtc}` call. Note that your error is due to packages you use that define `\ifpdf` which is obsolete (the result is that `\ifpdf` is defined twice); these packages should be changed to use `ifpdf` package instead.

B.2 pdfendlink error

Occasionally (for some LaTeX distributions) this hyper-linked bibTeX style may lead to **compilation errors** (`pdfendlink ended up in different nesting level ...`) if a reference entry is broken across two pages (due to a bug in `hyperref`). In this case, make sure you have the latest version of the `hyperref` package (i.e. update your LaTeX installation/packages) or, alternatively, revert back to `\bibliographystyle{abbrv-doi}` (at the expense of removing hyperlinks from the bibliography) and try `\bibliographystyle{abbrv-doi-hyperref}` again after some more editing.